

Chapter: How to Find a Layer in a Cake You Cannot See

*A Feynman-style lecture on the first-place solution to **ROGII** — Wellbore Geology Prediction (write-up).*

This chapter is about a Kaggle contest in shale-oil geosteering. The winning code lives in this repository. The six numbered folders under `reference_results/` are frozen snapshots of the six neural families that were blended into the \$25,000 first-place submission. Unless a sentence says otherwise, line numbers refer to the latest snapshot, `reference_results/0803_V2/`, which this package treats as the template.

1. The problem, said simply

Imagine a layered cake. The layers are rock: limestone, shale, a thin sweet spot of oil-bearing rock, more shale. A baker has already cut one vertical slice through the cake and written down, inch by inch, how “radioactive” each layer is. That slice is the **typewell**. The radioactivity is **gamma ray**, `GR`. It is not a chemistry assay. It is a fingerprint of lithology. Shale tends to read high; clean carbonate tends to read low. The fingerprint repeats whenever you meet the same layer again.

Now a drill bit is crawling *sideways* through the same cake. That is the **horizontal well**. At every foot of hole it reports:

- `MD` — how far the bit has traveled along the wellbore,
- `X`, `Y`, `Z` — where it is in space (`Z` is elevation, negative downward),
- `GR` — the gamma-ray fingerprint at that point,
- `TVT_input` — the geological “which-layer” coordinate, **but only for the first part of the well**.

At a moment called **Prediction Start**, `TVT_input` becomes empty. From there on you must guess `TVT`: which layer the bit is in. The contest scores you by RMSE in feet on those hidden `TVT` values.

That is the whole game. Not “predict a number from a table.” Match a sideways fingerprint against a vertical catalog, while the bit wanders up and down through dipping beds.

A row of a training horizontal file looks like this:

data/train/8995c945__horizontal_well.csv · lines 1–2

```
MD,X,Y,Z,ANCC,ASTNU,ASTNL,EGFDU,EGFDL,BUDA,TVT,GR,TVT_input
10254.0,3008769.24,1135030.04,-8170.44,-8454.76,-8563.74,-8661.63,-8823.61,-8874.0,-8979.59,1027
4.87,156.1464408070809,10274.87
```

The paired typewell is a vertical catalog sampled every half-foot of `TVT`:

data/train/8995c945__typewell.csv · lines 1–3

```
TVT,GR,Geology
10262.45,179.66,
10262.95,187.6,
```

Training has 773 such pairs. Formation tops (ANCC , ASTNU , ...) and the true TVT exist only in train. At test time you get trajectory, GR , the visible prefix of TVT_input , and the typewell. The metric is RMSE on tvt . A naive row-wise model — XGBoost on [X,Y,Z,MD,GR] — lands around **14 ft**. Physics trackers get you into the **8–10 ft** range. The public notebooks of early summer 2026 sat around **7.5 ft**. The winning private score was **5.639 ft**.

2. What TVT actually is

TVT is a terrible name for a newcomer, so let us rename it in our heads.

Z is Euclidean height. If the bit goes down two feet in space, Z drops two feet. TVT is **stratigraphic** height: how far you have moved *through the layers*. If the beds dip, crawling horizontally can still change TVT . If the beds are flat and you drill perfectly level, TVT barely moves even though MD grows by thousands of feet.

Geologists like the combination

$$S = TVT + Z$$

because S is a “structure surface”: it stays nearly constant if you travel along a bed, and it jumps if you cross a fault or a steep dip. The winning code uses this constantly. The geographic prior predicts S , then subtracts Z to get TVT . The particle filter tracks S as a particle position and subtracts Z to report TVT . The last smoother even Savitzky–Golay-filters TVT + Z and subtracts Z again.

One more fact, and then the whole recipe becomes obvious. **All TVT in this pipeline is relative to the last visible TVT_input .** That last known value is the anchor, TVT0 . After Prediction Start you are not guessing “the bit is at 10,400 ft of TVT.” You are guessing “the bit is 3.2 ft of TVT away from where we last knew it.” The config says so in so many words:

reference_results/0803_V2/seq_NN_cfg.py · lines 83–97

```
# Sequence construction.
downsample = 32
prefix_len = 1024
target_len = 10000
raw_len = prefix_len + target_len
num_bins = raw_len // downsample + 1
# ...
typewell_window = 100.0
typewell_len = 400
```

And the window builder copies the last 1,024 visible feet plus the next 10,000 target feet, subtracting the anchor from everything:

reference_results/0803_V2/seq_NN_dataset.py · lines 756–771

```
tvf_input = horizontal["TVF_input"]
tvf0 = float(tvf_input[last_seen_idx])
x0 = float(horizontal["X"][last_seen_idx])
y0 = float(horizontal["Y"][last_seen_idx])
z0 = float(horizontal["Z"][last_seen_idx])
md0 = float(horizontal["MD"][last_seen_idx])
...
prefix_start = max(0, last_seen_idx + 1 - prefix_len)
prefix_size = last_seen_idx + 1 - prefix_start
prefix_dest_start = prefix_len - prefix_size
suffix_end = min(row_count, last_seen_idx + 1 + target_len)
```

Downsampled by 32, that 11,024-foot window becomes **345** bins along the well. The typewell is cropped to **±100 ft around TVF0**, at 0.5 ft, which is **400** bins. The network therefore sees a **345 × 400 image**. That is not a coincidence. It is the whole idea.

3. Why ordinary regression is the wrong question

If you ask a neural net “what is **TVF** at this row?”, you are asking it to output a real number. There are two problems.

First, the right answer is almost never unique from one **GR** sample. A gamma-ray of 160 API units appears in several layers. The *sequence* of **GR** — the wiggles, the peaks, the missing-value gaps — is what identifies the layer. That is a 2-D matching problem, not a 1-D regression.

Second, when two alignments are almost equally good, a squared-error regressor averages them and lands in the empty rock in between. A geosteerer never does that. A geosteerer says “it is probably this peak, maybe that one,” and keeps a *distribution* over alignments.

So the winning author, Ruby, did the thing that looks slightly crazy until you have thought about it: **treat the job as image segmentation of an alignment grid**.

4. The 2-D alignment picture

Paint a picture.

Along the horizontal axis (well direction) put 345 positions: the visible prefix plus the hidden suffix, coarsened by 32 ft. Along the vertical axis (typewell direction) put 400 possible layers, a 200-foot

window around the anchor. At each pixel (i, j) you can ask: “if bin i of the well were sitting on typewell layer j , would the gamma rays match?”

The model’s main head is a 1×1 convolution that, for each well-bin, produces 400 logits. Softmax along the typewell axis turns those logits into a probability distribution over layers. The expected layer, using the known typewell TVT grid, is the predicted TVT .

reference_results/0803_V2/seq_NN_models.py · lines 625–716

```
self.alignment_head = nn.Conv2d(unet_emb_dim, 1, kernel_size=1)
typewell_tvt_axis_raw = torch.linspace(
    -typewell_window + typewell_window / typewell_len,
    typewell_window - typewell_window / typewell_len,
    typewell_len,
    dtype=torch.float32,
)
...
def _regression_from_logits(self, logits):
    prob = F.softmax(logits, dim=-1)
    tv_t_axis = self.typewell_tvt_axis.to(device=prob.device, dtype=prob.dtype)
    return (prob * tv_t_axis.reshape(1, 1, -1)).sum(dim=-1)
```

Notice: there is no separate “regression network.” Regression is the *barycenter* of the alignment distribution. If the probability mass is a sharp peak on the right layer, the expected TVT is that layer. If the mass is split, you at least know the model is uncertain — and the training loss can punish the split instead of quietly averaging it.

At inference the 345 bin predictions are stamped back onto the original one-foot rows by repeating each bin 32 times, then adding back TVT0 :

reference_results/0803_V2/seq_NN_train.py · lines 2090–2146

```
def expand_target_prediction(bin_pred, meta, cfg):
    ...
    for raw_idx in range(kept_len):
        bin_idx = (cfg.prefix_len + raw_idx) // cfg.downsample
        target_pred[raw_idx] = bin_pred[bin_idx] * std + mean
    ...

def target_value_to_tvt(target_value, meta, cfg):
    ...
    return tv_t0 + target_value
```

The network thinks in *relative, binned, normalized* TVT. The submission thinks in *absolute, one-foot* TVT. Those two lines are the dictionary between them.

5. The target is a smear, not a spike

If you trained with a one-hot “the true layer is bin 217,” the network would be yelled at for putting mass on bin 216, even though 216 is half a foot away and, for RMSE, almost as good. So the target is a

Gaussian bump centered on the true relative TVT, with $\sigma = 1.25$ ft, then normalized along the typewell axis.

reference_results/0803_V2/seq_NN_cfg.py · lines 594–595

```
alignment_target_mode = "exp_smooth" # "nearest2"/"exp_smooth"/"laplace"
alignment_exp_smooth_sigma = 1.25
```

reference_results/0803_V2/seq_NN_dataset.py · lines 1262–1278

```
def _make_typewell_exp_smooth_target_probs(tvt_target, target_mask, tvt0, cfg):
    ...
    sigma = float(getattr(cfg, "alignment_exp_smooth_sigma", 1.0))
    ...
    target_rel = tvt_target[valid].astype("float64", copy=False) - float(tvt0)
    target_rel = np.clip(target_rel, axis[0], axis[-1])
    diff = axis[None, :] - target_rel[:, None]
    target_probs = np.exp(-0.5 * np.square(diff / sigma))
    target_probs /= target_probs.sum(axis=1, keepdims=True)
```

That is label smoothing with a physical length scale. The 1.25 ft is not magic; it is “a little bigger than the 0.5 ft typewell sampling, a little smaller than the 32 ft well-axis bin.” Close enough that the loss still cares which peak you picked, smeared enough that neighboring bins are not treated as mortal enemies.

There are two other target recipes in the same file — linear interpolation onto the two nearest bins (`nearest2`) and a Laplace bump — but the submitted families all use `exp_smooth`.

6. Three losses, one picture

The loss is a weighted sum of three terms. The weights are tiny-looking fractions that add to 0.1; they are just a historical scaling.

reference_results/0803_V2/seq_NN_cfg.py · lines 600–608

```
UNET_loss_weights = {
    "regression": 0.05/3,
    "offset": 0.0,
    "alignment": 0.05*4/3,
    "CDF": 0.0,
    "GR_penalty": 0.05/3,
    "dS_penalty": 0.0,
    "UNET_GR_RMSE": 0.0,
}
```

So alignment gets four times the weight of either of the other two live terms. Let us look at each.

Alignment is soft cross-entropy: the Gaussian target times log-softmax of the logits, masked to suffix bins.

reference_results/0803_V2/seq_NN_train.py · lines 1206–1209

```
def masked_typewell_soft_cross_entropy(logits, target_probs, target_mask, loss_weight=None,
loss_scale_weight=None):
    log_prob = F.log_softmax(logits.float(), dim=-1)
    per_bin_loss = -(target_probs.float() * log_prob).sum(dim=-1)
    return _weighted_masked_mean(per_bin_loss, target_mask, loss_weight, loss_scale_weight)
```

This is the main teacher. It says: put probability mass where the true alignment is.

Regression is Huber loss on the expected TVT (the barycenter we already met). RMSE is the contest metric, but Huber is less hysterical about the occasional 30-foot mis-pick, which do happen when GR is missing or a fault jumps the surface.

reference_results/0803_V2/seq_NN_train.py · lines 1348–1357

```
if weights["regression"] != 0.0:
    regression_loss = masked_regression_loss_values(
        pred,
        target,
        target_mask,
        "huber",
        cfg,
        ...
    )
```

GR penalty is the sneaky one. After softmax you have, at each well-bin, a probability over typewell layers. Multiply that probability by the squared GR mismatch of each candidate layer, and take the expectation. In English: *even if you have not yet been told the true TVT, do not put mass on layers whose gamma ray looks nothing like what the bit is seeing.*

reference_results/0803_V2/seq_NN_train.py · lines 1229–1232

```
def masked_gr_penalty_loss(logits, gr_error, target_mask, loss_weight=None,
loss_scale_weight=None):
    prob = F.softmax(logits.float(), dim=-1)
    per_bin_loss = (prob * gr_error.float()).sum(dim=-1)
    return _weighted_masked_mean(per_bin_loss, target_mask, loss_weight, loss_scale_weight)
```

The GR-gap image itself is built in the dataset, clipped so a single crazy mismatch cannot dominate:

reference_results/0803_V2/seq_NN_dataset.py · lines 2493–2510

```
def _make_gr_penalty_error(horizontal_map, typewell_grid, cfg, gr_norm=None):
    ...
    penalty = np.square(((tw_gr[None, :] - gr[:, None]) / diff_std).astype("float32"))
    clip = float(getattr(cfg, "GR_penalty_clip", 25.0))
    if clip > 0.0:
        penalty = np.minimum(penalty, np.float32(clip))
```

The write-up calls this “a small additional improvement.” That is honest. Alignment does the heavy lifting. Huber keeps the expected path near the metric. GR penalty is a weak physics regularizer that costs almost nothing at train time and occasionally saves a well.

All three are assembled in `_UNET_loss_single` (`seq_NN_train.py` lines 1300–1459). Several other losses exist in the same function — CDF matching, surface-curvature `dS` penalty, an auxiliary GR-RMSE head — and they are all weighted to zero in the winning configs. The code is a laboratory. The recipe is the three terms above.

7. Painting the 345×400 image

A blank 345×400 grid is not enough. The U-Net needs channels: facts about each pixel that a convolutional eye can use. The default channel list is the beating heart of the recipe.

reference_results/0803_V2/seq_NN_cfg.py · lines 738–766

```
UNET_STATIC_CHANNELS = (
    "tw_gr",
    "gr",
    "gr_abs_diff",
    "gr_isnan_rate",
    "gr_std",
    "gr_first_last_delta",
    "gr_slope",
    "gr_quadratic_a",
    "gr_quadratic_b",
    "gr_quadratic_c",
    "gr_quadratic_rmse",
    "tw_gr_is_nan",
    "tw_tvt_rel",
    "seen_tvt_rel",
    "tw_seen_tvt_abs_diff",
    'geo_tvt_diff',
    ...
)
```

There are four kinds of paint.

7.1 Typewell paint (the catalog)

`tw_gr` is the typewell gamma-ray, **broadcast down every well-bin**, so column j of the image is “what layer j is supposed to look like.” `tw_tvt_rel` is the relative TVT of that column. `tw_gr_is_nan` marks catalog holes. Construction is in `_make_unet_static_input` (`seq_NN_dataset.py` around lines 2092–2148 and 2244–2251).

7.2 Horizontal-well paint (what the bit sees)

`gr` is the well’s own gamma ray, **broadcast across every typewell column**, so row i is “what the bit actually saw in this 32-foot bin.” Then come the texture statistics, all computed on the raw one-foot samples and then downsampled:

CHANNEL	WHAT IT IS
<code>gr_isnan_rate</code>	fraction of missing GR in the bin
<code>gr_std</code>	spread of GR in the bin
<code>gr_first_last_delta</code>	last finite GR minus first
<code>gr_slope</code>	GR versus MD slope
<code>gr_quadratic_a,b,c,rmse</code>	a local parabola fit to GR inside the bin

reference_results/0803_V2/seq_NN_dataset.py · lines 1005–1058

```
def _downsample_horizontal_feature(raw, gr_nan, raw_has_row, name, cfg):
    if name == "gr_nan_rate":
        return _downsample_rate(...)
    if name == "gr_std":
        return _downsample_std(...)
    ...
    if name == "gr_first_last_delta":
        return _downsample_first_last_delta(...)
    if name == "gr_slope":
        return _downsample_slope(...)
    if name in GR_QUADRATIC_CHANNELS:
        return _downsample_gr_quadratic(...)[name]
```

The quadratic fit is not decoration. A 32-foot bin can hide a peak, a ramp, or a flat. Mean GR alone cannot tell those apart. A parabola can.

7.3 Comparison paint (the actual matching cue)

`gr_abs_diff` is $|GR_{well} - GR_{typewell}|$, the thing a geologist’s eye looks at.

`tw_seen_tvt_abs_diff` is $|TVT_{typewell} - TVT_{visible\ prefix}|$, which is only informative on the prefix — but the prefix is where the network learns what “a good match” looks like, and that lesson transfers to the suffix.

7.4 Motion paint (how the well is moving)

`seen_tvt_rel` is the already-known relative TVT on the prefix. On families without a geographic prior, `z_diff` — the change in elevation from one row to the next — is the cheap physics: if you go down, and the beds are flat, TVT should go up. On families *with* a geographic prior, `geo_tvt_diff` replaces that guess with a neighbor-informed one. We will come back to it.

The constructor that turns those names into a `(C, 345, 400)` tensor is `_make_unet_static_input` in `seq_NN_dataset.py`, starting at line 2092. Every later trick — particle-filter heatmaps, extra geo channels — is just more names on the same list.

8. The typewell lies a little, so we calibrate it

The typewell was logged with a different tool, at a different time, often with a different borehole environment. Its GR amplitude is not on the same scale as the horizontal well. If you match them raw, you match the *shape* and fight the *offset*.

The fix is almost embarrassingly simple, and it is one of the highest-leverage lines in the pipeline. Take the visible prefix of the horizontal well, keep only samples within 100 ft of TVT0, bin them to a quarter-foot TVT grid, and **overwrite the overlapping region of the typewell with a blend of that pseudo-curve and the original typewell**.

reference_results/0803_V2/seq_NN_dataset.py · lines 6220–6286

```
def GR_calibration(h_df, v_df, typewell_power=1.0, blend_weight=0.7):
    """Blend a visible-prefix pseudo curve with a calibrated raw Typewell.
    ...
    """
    ...
    tvt0 = seen_h_df["TVT_input"].iloc[-1]
    refer_cond = seen_h_df["TVT_input"].between(tvt0 - 100, tvt0 + 100)
    ...
    refer_df["TVT"] = (refer_df["TVT_input"] * 4).round(0) / 4
    refer_df = refer_df.groupby("TVT")["GR"].mean().reset_index().sort_values("TVT")
    ...
    v_df.loc[cond, "GR_cali"] = np.interp(...)
    ...
    v_df.loc[cond, "GR_cali"] = (
        blend_weight * v_df.loc[cond, "GR_cali"]
        + raw_weight * calibrated_raw_gr.loc[cond]
    )
```

Default blend is 70% “what this well’s own prefix says the catalog should look like” and 30% “the original typewell.” Config:

reference_results/0803_V2/seq_NN_cfg.py · lines 255–257

```
typewell_calibration_with_seen = True
typewell_calibration_power = 1.0
typewell_calibration_blend_weight = 0.7
```

After this, `gr_abs_diff` is comparing apples to apples, at least near the anchor. Far from the anchor the original typewell still governs, which is correct: the prefix cannot tell you about layers you have not yet drilled.

9. Neighbors in the field: geology is continuous

Wells in a shale play are not random points in a plane. They are pads. A dozen laterals may sit a few hundred feet apart, chewing through the *same structure*. If your neighbor, 400 feet to the north, was 8 feet of TVT above the landing zone at this XY, you probably are too.

The geographic prior is a small physics model that never sees GR. It asks only: given nearby wells' known $S = \text{TVT} + Z$ along their paths, what is S along mine?

The method name is `idw_dS_xy_wls`. In English:

1. Find the 12 nearest training wells by centroid XY (`GeoPriorConfig.neighbor_wells = 12`).
2. Treat their path as samples of the structure gradient $\nabla S = (\partial S / \partial x, \partial S / \partial y)$.
3. At each query point, inverse-distance-weight those samples, then solve a **ridge-regularized, Tukey-robust weighted least squares** for the local gradient, with an anisotropy of 50° and ratio 1.3 so that “along the play’s strike” is not treated the same as “across it.”
4. Integrate $dS = \nabla S \cdot (dx, dy)$ along the query well, starting from the known prefix.

reference_results/0803_V2/seq_NN_cfg.py · lines 39–64

```
class GeoPriorConfig:
    method: str = "idw_dS_xy_wls"
    neighbor_wells: int = 12
    point_neighbors: int = 112
    idw_power: float = 2.0
    point_neighbor_metric: str = "anisotropic"
    anisotropy_angle_deg: float = 50.0
    anisotropy_ratio: float = 1.3
    ...
    wls_ridge_frac: float = 0.01
    wls_robust_mode: str = "tukey"
```

The WLS solve, with a fallback to plain IDW when the local geometry is rank-deficient (all neighbors in a line, for example):

reference_results/0803_V2/seq_NN_geo_prior.py · lines 414–453

```
def _solve_weighted_dS_xy_wls_once(...):
    ...
    ridge = np.maximum(trace * float(cfg.wls_ridge_frac), 1e-12)
    ...
    use_wls = solvable & (eig_ratio ≥ float(cfg.wls_min_eig_ratio)) & grad_ok
    beta[~use_wls] = fallback[~use_wls]
```

And the integration along the well:

reference_results/0803_V2/seq_NN_geo_prior.py · lines 734–737

```
ds = raw_grad[:, 0] * (query_well.x[raw_idx] - query_well.x[prev_idx])
ds += raw_grad[:, 1] * (query_well.y[raw_idx] - query_well.y[prev_idx])
out[raw_idx] = np.cumsum(ds, dtype=np.float64).astype(np.float32)
```

Standalone, this prior is mediocre — the write-up quotes about **11.4 ft** RMSE, and the training logs print fold geo-prior RMSEs around 11–13 ft. That is not a contest-winning number. It is a *channel*. Once you have `S_rel`, you can form the TVT increment the neighbors imply:

reference_results/0803_V2/seq_NN_dataset.py · lines 1563–1569

```
def _geo_tvt_diff_from_s_rel(geo_s_rel, z_diff, suffix_mask, cfg):
    values = _geo_dS_from_s_rel(geo_s_rel, suffix_mask, cfg) - np.asarray(
        z_diff,
        dtype=np.float32,
    )
```

because $dTVT = dS - dZ$. That tensor, `geo_tvt_diff`, is broadcast across the typewell axis and handed to the U-Net. The network is free to trust it, ignore it, or use it only when GR is ambiguous. That is much healthier than *replacing* the GR match with the neighbor guess.

Two warnings, which the winner actually respected.

Leakage. The prior for a validation well must not see that well’s own hidden TVT, and must not see other validation wells. The prior is built inside each fold from the training wells of that fold (`seq_NN_geo_prior.py`, `make_geo_prior_for_wells`). Geographic CV (section 13) exists for the same reason: a random split would put a pad-mate in train and another in val, and the prior would look like a genius.

Wells with no neighbors. About 10% of wells sit alone. The neighbor gradient is then garbage. The write-up’s inference rule: if neighborhood statistics exceed the 95th percentile of training, *drop the XY-neighbor families* and use the `z_diff` families instead. In this repo that split is exactly the difference between `0801_V1` (`z_diff`) and `0801_V2` (`geo_tvt_diff`):

reference_results/0801_V2/seq_NN_cfg.py · lines 3040–3048

```
v2_cfg = deepcopy(v1_cfg)
v2_cfg.unet_static_channels = _with_features(
    _without_features(v2_cfg.unet_static_channels, ("z_diff",)),
    ("geo_tvt_diff",),
)
...
("submit_ver_0801_V1", v1_cfg),
("submit_ver_0801_V2", v2_cfg.refresh()),
```

The ensemble notebook (not bundled here; it lives at [submit-reproduce](#)) applies different family weights to “general” wells versus “no reliable XY” wells. We will quote those weights in section 14.

10. The particle filter: a drunken walker that likes matching GR

A particle filter is what you use when you have a pretty good idea of the *dynamics* and a noisy idea of the *observations*.

Dynamics: the structure surface `S` changes slowly as you drill. Most steps, `S` moves by a fraction of a foot. Rarely, a fault jumps it by several feet. Observation: at the current `TVT = S - Z`, the typewell GR should look like the bit GR.

The simplest version in the repo is readable in one screen. It is not the production heatmap, but it is the same animal:

reference_results/0803_V2/seq_NN_data_prep.py · lines 285–365

```
def run_particle_filter(hw, tw, n_particles=500, seed=42):
    """Conservative PF. Returns (predictions_array, total_log_likelihood)."""
    ...
    pos = ls + 3.0 * rng.standard_normal(N) # wider init spread helps wells with abrupt TVT
    shift at PS
    rate = ir + 0.01 * rng.standard_normal(N)
    ...
    for i in range(len(ev)):
        dm_step = max(md_v[i] - prev_MD, 1.0)
        rate = MOM * rate + VN * rng.standard_normal(N)
        pos = pos + rate * dm_step + PN * rng.standard_normal(N)
        tv_t_p = pos - z_v[i]
        ...
        eg = np.interp(tv_t_p, tw_tv_t, tw_gr)
        d = (gr_v[i] - eg) / gs
        lk = np.exp(-0.5 * np.minimum(d**2, 600.))
        w = w * lk
        ...
        n_eff = 1.0 / (w**2).sum()
        if n_eff < RESAMP * N:
            ... # systematic resample, then jitter
```

Read it as a story. Each particle is a hypothesis: “the surface is here, and it is drifting at this rate.” Each foot, every particle takes a small random step. Then it is weighed by how well the implied typewell GR matches the bit. When too many particles have died (effective sample size too small), you clone the survivors and jiggle them. The reported TVT is the weighted average.

Standalone, a 128-seed likelihood-weighted ensemble of this tracker is about **7.4 ft** RMSE. That would have been a respectable mid-competition score by itself. By the time the U-Net was strong, the PF no longer helped the *best single model*. Ruby kept it anyway, as extra channels, for diversity.

The production tracker is a Numba kernel, `_pf_heatmap_kernel`, that dumps particle occupancy into the same 345×400 grid the U-Net uses. A second kernel, `_pf_heatmap_ffbsi_kernel`, runs **forward-filtering backward simulation** (FFBSi): after filtering you sample smoothed trajectories backwards through the stored particle clouds. Filtering is causal; FFBSi gets to use the future GR of the well, which a geosteerer in the field does not have, but a Kaggle model looking at a finished log does.

reference_results/0803_V2/seq_NN_cfg.py · lines 104–192

```
PF_heatmap_n_particles = 500
PF_heatmap_n_seeds = 128
...
PF_heatmap_jump_prob = 0.000075
PF_heatmap_jump_sd = 7.0
...
PF_heatmap_ffbsi_mode = "fallback"
PF_heatmap_ffbsi_n_paths = 64
```

Five slightly different PF “personalities” are mixed — a stiff smoother, a rare-big-jumper, a wide initializer, and so on (`PF_heatmap_profile_mixture_spec` , lines 195–244). The output channels that actually enter the winning `0803_V2` family are:

- `pf_particle_density_prob` — the occupancy heatmap,
- `pf_prob_ffbsi` — the smoothed occupancy heatmap,

plus, in the same family, the accumulated geo channels `geo_tvt_abs_diff` and `geo_tvt_rel`. Registry:

reference_results/0803_V2/seq_NN_cfg.py · lines 3018–3034

```
def make_seq_requested_ablation_cfgs():
    submit_v1_cfg = make_static_channel_add_cfg(
        static_add=("pf_particle_density_prob", "pf_prob_ffbsi"),
    )
    submit_v2_cfg = make_static_channel_add_cfg(
        static_add=(
            "pf_particle_density_prob",
            "pf_prob_ffbsi",
            "geo_tvt_abs_diff",
            "geo_tvt_rel",
        ),
    )
    ...
    ("submit_ver_0803_V2", submit_v2_cfg),
```

The write-up’s remark is worth repeating: *once augmentations and a pretrained backbone were strong, PF channels stopped helping the best single model, and were kept for ensemble diversity*. That is a grown-up thing to do. A 7.4 ft tracker is still a different *kind of error* from a 4.8 ft ConvNeXt.

11. Teaching the network with fake wells

There are only 773 training laterals. A ConvNeXt pretrained on ImageNet has seen a million photographs and is hungry. If you feed it the same 773 wells unperturbed, it memorizes pad geometry and GR quirks. The winner’s answer is to **simulate new laterals that are physically legal**.

The two augmentations the write-up calls “the most important” are Z-shift and GR transform. They are not the only ones, but they are the ones that change the *meaning* of the picture.

11.1 Z-shift: move the well through the cake, keep the surface

The docstring is a complete lecture:

reference_results/0803_V2/seq_NN_dataset.py · lines 5243–5245

```
'''
    x,y stay the same, shift tvf while hold (tvf+z) unchanged
'''
```

Hold $S = \text{TVT} + Z$ fixed, slide **TVT** and **Z** in opposite directions. The well is now in a different layer at the same map location. Then **rebuild GR from the typewell at the new TVT, and add a residual** so the simulated log is not a perfectly clean copy of the catalog:

reference_results/0803_V2/seq_NN_dataset.py · lines 5369–5375

```
# create new GR by keep noise component unchanged
GR=well_data['horizontal']['GR']
v_GR=well_data['typewell']['GR']
v_TVT=well_data['typewell']['TVT']
sim_matched_GR=np.interp(TVT_sim, v_TVT, v_GR, left=np.nan, right=np.nan)
matched_GR=np.interp(TVT, v_TVT, v_GR, left=np.nan, right=np.nan)
base_noise=GR-matched_GR
```

The residual is not copied blindly. The live noise mode is **TVT_bias_v2**: smooth, TVT-correlated GR corruption with a realistic amplitude mix. On top of that, 5% of samples get Laplace “faults” (local jumps in TVT with **S** intentionally *not* updated, so the geo prior is wrong — a gift to a network that must learn when not to trust neighbors). The trend of the new TVT path is itself a bootstrap of real suffix differences from other wells (**sample_model='diff_block_bootstrap'**). Apply probability: **0.85**.

reference_results/0803_V2/seq_NN_cfg.py · lines 267–282

```
'z_shift':{
    'apply_prob':0.85,
    ...
    'sample_model':'diff_block_bootstrap',
    ...
    'z_shift_range':40,
    'fault_ratio':0.05,
    'fault_max_num':3,
    'fault_std':1.75,
    'noise_mode':{'TVT_bias_v2':1.0},
```

This is why the model generalizes off the training pads. It has spent most of its life in *nearby, physically possible* cakes, not in the 773 original ones.

11.2 GR transform: the catalog is allowed to be the wrong brightness

reference_results/0803_V2/seq_NN_dataset.py · lines 6031–6062

```
def GR_trf(self, well_data, cfg=None):
    ...
    a = np.float32(np.random.normal(1.0, a_std))
    b = np.float32(np.random.normal(0.0, b_std))
    ...
    out = (a * out + b).astype(np.float32)
```

Affine warp of typewell GR, sometimes a little smoothing or sharpening, apply probability 0.6. Combined with the 70/30 calibration of section 8, this is how the network learns that “the catalog is a cousin of the well log, not a twin.”

11.3 The rest of the zoo

`WellDataSimulator.process` (`seq_NN_dataset.py` lines 6103–6202) is the ordered pipeline. Besides Z-shift and GR transform it may, with smaller probabilities:

- reverse the path (drill the well backwards),
- stretch MD,
- cut the tail,
- jitter typewell GR with a coherent residual,
- drift the whole typewell TVT axis by ~1 ft (`typewell_drift`, 30%),
- randomly mask 10% of U-Net channels (`channel_mask_2d`, 70% of samples),
- slightly rotate/shift the PF heatmap so the network cannot memorize it.

The last two are the deep-learning equivalents of “do not let the model stare at one cheat sheet.” If a PF channel is masked, the alignment head still has to work from GR. If the PF heatmap is shifted a few feet, the network learns to *correct* a biased tracker rather than copy it.

12. The eye: a ConvNeXt U-Net looking at a tiny image

The backbone is not a 1-D LSTM. It is a 2-D U-Net whose encoder is **ConvNeXt-Small pretrained on ImageNet-12k then fine-tuned on ImageNet-1k at 384 px**, loaded through `timm`:

reference_results/0803_V2/seq_NN_cfg.py · lines 840–843

```
unet_cfg = {
    "unet_arch": "convnext_small",
    "unet_pretrained": None,
    "unet_timm_model_name": "hf_hub:timm/convnext_small.in12k_ft_in1k_384",
```

(`unet_pretrained: None` means “download weights when training, skip the download when unpickling an already-trained submit model.”)

The wrapper, `PretrainedUNet2d` in `seq_NN_pretrained_unet.py`, takes the four ConvNeXt stages, replaces `LayerNorm` with `BatchNorm` so small-batch geosteering tensors behave, and attaches a bilinear `ResBlock` decoder back to 345×400 . Stem stride is `(2, 4)`: coarser along the typewell axis than along the well, because 400 typewell bins of 0.5 ft are a different physical scale from 345 well bins of 32 ft.

Why a vision model? Because the alignment grid is an image. A peak in `gr_abs_diff` that tilts as you move along the well is a dipping bed. A horizontal dark stripe is a layer the bit is tracking. ConvNeXt’s inductive bias — local filters, hierarchical features — is exactly the bias of a geologist looking at a correlation panel in StarSteer.

Training details, all in the same CFG :

- AdamW, learning rate 2×10^{-4} , cosine decay, 200 epochs, stop after epoch 170 if validation RMSE has not improved for 50 epochs.
- Batch size 16, AMP bfloat16, EMA decay 0.99, channels-last memory format.
- **Not** fully deterministic CUDA: the historical run disabled `deterministic` because strict kernels made the ConvNet much slower. Reproduction therefore matches *in distribution*, not byte-for-byte.

reference_results/0803_V2/seq_NN_cfg.py · lines 727–734

```
amp_dtype = "bfloat16"
ema_enabled = True
ema_decay = 0.99
# Seeds are always fixed in seq_NN_train.py. This flag additionally enables
# strict deterministic CUDA kernels, which is reproducible but much slower
# for the 2D ConvNet.
deterministic = False
channels_last_2d = True
```

After the network speaks, one last geological low-pass filter is applied. `TVT + Z` should be a smooth surface. Savitzky–Golay with a 577-foot window and polynomial order 1, blended 72.6% of the way toward the smoothed surface, kills jitter without flattening real faults:

reference_results/0803_V2/seq_NN_train.py · lines 2168–2190

```
def apply_pred_sg_smooth(df, cfg):
    ...
    window = int(getattr(cfg, "pred_sg_smooth_window", 577))
    polyorder = int(getattr(cfg, "pred_sg_smooth_polyorder", 1))
    blend = float(getattr(cfg, "pred_sg_smooth_blend", 0.7261960515627183))
    for pos in out.groupby("well_id", sort=False).indices.values():
        ...
        surface = raw[pos] + z[pos]
        candidate[pos] = _savgol_1d(surface, window=window, polyorder=polyorder) - z[pos]
        final = raw + blend * (candidate - raw)
```

That is the same `S = TVT + Z` idea, now used as a post-process rather than a prior.

13. How you know you are not kidding yourself

The public leaderboard was about 50–60 wells. It was, in the winner’s words and in the community’s consensus, close to noise. XY-neighbor models looked *worse* on the public board than they did in local CV; the winner attributed the gap to inconsistent labels on a handful of public wells and **trusted CV**, which said the neighbor channels were worth about 0.3 ft.

The CV itself is not a random GroupKFold. Wells are clustered in XY by KMeans (20 clusters) and then stratified into 5 folds, repeated 3 times with different split seeds — 15 models per family.

reference_results/0803_V2/seq_NN_train.py · lines 380–421

```
def make_geo_stratified_folds(well_ids, cfg, log, *, split_seed=None, repeat=None):
    ...
    xy = split_df[["horizontal_avg_X", "horizontal_avg_Y"]].to_numpy(dtype=float)
    ...
    kmeans = KMeans(n_clusters=n_clusters, random_state=kmeans_seed, n_init="auto").fit(xy)
```

reference_results/0803_V2/seq_NN_cfg.py · lines 660–664

```
fold_count = 5
cv_repeats = 3
...
cv_kmeans_clusters = 20
```

The map of well centroids is rebuilt, not trusted from the original PNG sidecars, by `generate_train_geo_map.py`. Geographic clustering does two jobs at once: it makes validation resemble “a new pad in a known play,” and it keeps the geo-prior honest.

14. Six families, two well types, one blend

A single 15-fold ConvNeXt is already a strong model. The winner trained **six** of them, on purpose, with different extra channels, and blended.

SNAPSHOT	WHAT WAS ADDED ON TOP OF THE GR IMAGE	ARCHIVED OOF RMSE	ROLE
0719_V1	<code>z_diff</code> only (default <code>CFG()</code> of that day)	5.0910 ft	no-XY workhorse
0724_V1	<code>z_diff</code> and <code>geo_tvtt_diff</code>	4.8586 ft	XY workhorse
0729_V3	PF density + FFBSi TVT (<code>submit_ver_0729_V3</code>)	5.5360 ft	PF diversity, no-XY
0801_V1	<code>z_diff</code> (later augmentations)	5.1668 ft	no-XY workhorse
0801_V2	<code>geo_tvtt_diff</code> instead of <code>z_diff</code>	4.8045 ft	best single, XY
0803_V2	PF heatmaps and accumulated geo TVT	5.0055 ft	XY + PF diversity

Those OOF numbers are not folklore. They are the last line of each archived `seq_nn.log`, for example:

reference_results/0724_V1/seq_nn.log · lines 668–668

```
[2026-07-25 07:33:53] OOF RMSE: 4.8586 | FOLD RMSE:
5.1051/4.3285/5.6588/4.4691/4.9367/4.8372/4.2926/5.7035/5.4927/5.3233/5.2619/5.4571/5.3707/4.973
4/5.2158 (mean:5.0951 +- std:0.4380)
```

reference_results/0801_V2/seq_nn.log · lines 750–750

```
[2026-08-02 16:41:12] OOF RMSE: 4.8045 | FOLD RMSE:
5.2045/5.2762/4.8242/4.9221/5.1059/4.5426/5.1347/4.7563/5.3106/5.4979/4.5769/4.7466/5.7010/4.880
7/4.8813 (mean:5.0241 +- std:0.3216)
```

The reproduction wrapper’s only job is to pick the right snapshot and the right registry name:

```

RUN_RECIPES = {
    "0719_V1": {"mode": "common", "registry_name": None, "archived_f": 0},
    "0724_V1": {"mode": "common", "registry_name": None, "archived_f": 0},
    "0729_V3": {"mode": "registry", "registry_name": "submit_ver_0729_V3", "archived_f": 0},
    "0801_V1": {"mode": "registry", "registry_name": "submit_ver_0801_V1", "archived_f": 0},
    "0801_V2": {"mode": "registry", "registry_name": "submit_ver_0801_V2", "archived_f": 0},
    "0803_V2": {"mode": "registry", "registry_name": "submit_ver_0803_V2", "archived_f": 1},
}

```

common means “whatever CFG() defaulted to that day.” registry means “look up this name in SEQ_TRAIN_CFGS.” That is why 0801_V1 and 0801_V2 can share a snapshot directory: they are two rows of the same table.

The write-up’s blend, with public/private scores, was:

MODEL	WEIGHT ON ORDINARY WELLS	WEIGHT ON NO-XY WELLS	CV	PUBLIC	PRIVATE
Ensemble	—	—	4.627	5.980	5.639
0719_V1	0.07	0.40	5.09	5.648	6.130
0729_V3	0.00	0.20	5.53	6.202	6.768
0801_V1	0.07	0.40	5.16	5.723	5.884
0724_V1	0.28	0.00	4.86	6.095	5.831
0801_V2	0.28	0.00	4.80	6.166	5.937
0803_V2	0.28	0.00	5.00	6.185	5.778

Read the weights as a sentence. **On a well with neighbors, trust the three XY families equally and sprinkle a little of the older no-XY nets.** On a well without neighbors, **throw the XY families away** and lean on 0719_V1 and 0801_V1, with a dash of the PF-only 0729_V3.

Notice the public column. The best public single model is 0719_V1, which has *no* XY channels. The best private single model among the XY set is 0803_V2. If you had chased the public board you would have thrown away the thing that won. That is the whole morality play of this contest.

The six-family blend itself is not in this repository. Training one ID writes reproduction_outputs/<ID>/. The Kaggle notebook [submit-reproduce](#) consumes those six directories, applies the two-regime weights, and writes submission.csv.

15. The winning recipe, boiled down

If you had to tattoo the solution on one arm, it would be this.

1. **Ask the right question.** Not “what is TVT?” but “which typewell layer is this 32-foot bin sitting on?” That question is a 345×400 image.
2. **Train a distribution, report an expectation.** Gaussian-smoothed alignment as the target, cross-entropy as the main loss, Huber on the barycenter so RMSE stays honest, a little GR-gap penalty so impossible layers stay dark.
3. **Paint the image with matching cues**, not with raw XYZ. Calibrated typewell GR, well GR, their absolute difference, GR texture inside each bin, and the known prefix TVT.
4. **Give the network two extra opinions it is allowed to ignore:** a neighbor-well structure gradient, and a particle-filter occupancy heatmap. Do not *replace* GR matching with either one.
5. **Invent more wells than nature gave you**, by sliding laterals through the cake at constant S and by affine-warping the catalog GR.
6. **Use a 2-D vision backbone.** ConvNeXt-Small, ImageNet-pretrained, U-Net decoder, EMA, cosine AdamW.
7. **Validate like the map is real.** Geographic KMeans folds, three repeats, trust OOF over a 50-well public board.
8. **Blend families that fail differently**, and, at inference, switch the blend when the well has no neighbors.
9. **Smooth the structure surface**, not TVT itself.

Everything else in the 3,000-line config files — transformer U-Nets, two-stage correlation encoders, bidirectional ConvGRUs, CDF losses, random-search registries — is the laboratory in which that recipe was found. The submitted models do not use those paths.

16. Where to read the code, in order

If you want to walk the pipeline the way a well is walked, from bit to submission:

STEP	FILE	WHAT TO READ
Geometry of the image	seq_NN_cfg.py 83–97, 594–608, 738–766	bins, losses, channels
Anchor window	seq_NN_dataset.py 726–834	TVT0 , prefix/suffix copy
Soft alignment target	seq_NN_dataset.py 1262–1314	Gaussian / Laplace / nearest-2
Channel painting	seq_NN_dataset.py 2092–2280	_make_unet_static_input
Typewell calibration	seq_NN_dataset.py 6220–6286	GR_calibration
Z-shift / GR transform	seq_NN_dataset.py 5228–5375, 6031–6091, 6103–6202	the simulator
Geo prior	seq_NN_geo_prior.py 39–64 (config), 414–453, 637–737	WLS gradient, integrate dS
Particle filter	seq_NN_data_prep.py 285–385, then the Numba kernels at 3404 and 3998	toy PF, then production heatmap / FFBSi
U-Net + barycenter head	seq_NN_models.py 469–801	SeqUNet2DModel
ConvNeXt encoder	seq_NN_pretrained_unet.py 422–582	PretrainedUNet2d
Losses	seq_NN_train.py 803–823, 1206–1232, 1300–1459	weights and the three live terms
Geographic CV	seq_NN_train.py 380–421	KMeans + stratified folds
Decode and smooth	seq_NN_train.py 2090–2190	bins → feet, Savitzky–Golay on S
Which snapshot is which	seq_NN_main_reproduce.py 37–44, plus each snapshot's make_seq_requested_ablation_cfgs	the six-family roster
Archived scores	each reference_results/<ID>/seq_nn.log , last 00F RMSE line	what that family actually did

The contest page is **ROGII — Wellbore Geology Prediction**. The author’s own account of the same ideas, including the public/private table reproduced in section 14, is the **1st Place Solution write-up**.

The cake is still down there. The bit is still blind. The trick was never to guess a number. It was to ask, at every foot, which layer of a catalog this wiggle of gamma ray would rather be — and to let geology, neighbors, and a cloud of particles whisper, not dictate, the answer.